TripNest AI: Judge Demonstration Guide

Version 2.0 · 2 November 2026 · about 6 minutes. Everything shown is implemented and tested; limits are stated up front.

Before you start. Run the app locally (`README.md`) or open the live URL https://tripnest-ai-1rug.onrender.com. Keep `/docs` open in a second tab. If using a free-tier host, open the site a few minutes earlier so it is awake.

1. The problem (30 s)

Everyone arrives at the same famous places on the same weekends. Travellers cannot easily ask "how crowded will it be on my date, and where else could I go?" TripNest AI answers those two questions and nothing else, on purpose.

2. Journey 1: location-first discovery (120 s)

1. Home page: hero photograph, headline, search panel. Point out that no location prompt appeared.
2. Press **Use my current location** and deny it: the page asks for a city. Type **Coimbatore**, pick a Saturday, interest **Wildlife** or none, press **Explore Destinations**. Greeting: "Discover amazing places around Coimbatore."
3. **Popular places near you:** photo, state, distance, crowd chip, best time.
4. **Best time to visit:** four-slot timeline, calmest slot highlighted, hour window. Select another place and watch it change.
5. **Escape the Crowds:** each gem has a reason. Mention that gems are calm *and* lesser-known, not just unpopular.
6. Change the date to a weekday in the top bar: chips and best times move.

3. Journey 2: the map (60 s)

Show your position, the different markers for popular places and gems, then filters **Hidden gems** and **Low**. Open a popup (photo, level, best time) and press **View Details**.

4. Journey 3: one destination (60 s)

On the details page show the level, people with a likely range, the timeline, hourly chart and 7-day tiles. Change the time range to early morning to show the level drop. Open **About this prediction**: "ML prediction, Demonstration data, Not live", freshness, uncertainty, limitations.

5. Journey 4: privacy and phones (30 s)

Mention that location is only requested on a click (or if already allowed), is rounded to about 1 km and is not stored. Resize to a phone width: the same flow works with swipeable cards.

6. The ML and the evidence (90 s)

Open `/api/model` or the home page's "How good are the predictions?": - Gradient-boosted model, chronological train / validation / test split, two baselines. - Test MAPE **9.38%** against **31.79%** (same hour last week) and **27.45%** (weekday average). - **Say clearly:** the history is simulated, so this proves the pipeline works and beats simple baselines. It does **not** prove real-world accuracy. Swapping in real gate counts is one table and one command. - Show the test results (`docs/TEST_RESULTS.md`): backend suite and 101 backend and 24 browser tests, including the nine-step flow and four screen sizes.

Likely questions

Question	Honest answer
Is this real crowd data?	No. It is a model trained on a labelled demonstration history, because no verified public counts exist. The UI says so everywhere.
Is the recommender AI?	It is a transparent weighted score that consumes the ML predictions. We call only the crowd model machine learning.
Why trust 9.3% error?	Only as a pipeline check. The simulated data follows rules similar to what the model learns. Real-data accuracy is unknown until real counts are loaded.
Why show popular places at all?	Travellers expect them; each is labelled with its predicted crowd, and popularity is never part of the recommendation score.
Security?	No accounts or stored personal data; secrets only in environment variables and production refuses to start without them; server-side validation of every parameter; security headers with a content-security policy; non-root container. Not done: penetration test, rate limiting (all endpoints are read-only), dependency vulnerability scan.
Scalability and platform?	Stateless API, PostgreSQL in production, forecasts batched and cached in memory (single instance). Not load-tested.
Is it deployed?	Yes, on Render's free plan: https://tripnest-ai-1rug.onrender.com. It sleeps when idle, so the first load takes about 30 s. It uses a seeded SQLite file, not PostgreSQL.
What did you remove?	Itinerary planner, community, reviews, weather, accounts and admin dashboards, to focus on two features. The code is kept, unmounted.
What next?	Real data feeds (gate, parking, sensors), road-distance routing, weather and events as model inputs, an authority view.

Known limitations to volunteer

Simulated crowd history · assumed capacities · straight-line distance · photos and map need internet · Chrome-only browser tests · no load or accessibility-audit testing.